

Professores:

Celso Rodrigo Giusti

Daniel Manoel Filho

Felipe Augusto Stevanim Gregate

Marlon Palata Fanger Rodrigues

REACT • AULA 3

Componentes, Props e Listas



O que faremos hoje

 **Relembrar:** public/, src/, components/ e como os arquivos se conectam.

 **Componentização:** dividir a tela em pedaços reutilizáveis.

 **Props:** como o pai manda dados para o filho.

 **Listas:** .map(), key e .filter() por categoria.

 **Condicional:** mostrar ou esconder com && e ternário.

 **Fio condutor:** o cardápio do TechFood por categoria.

A estrutura do projeto

techfood-react/

- |— node_modules/ ← bibliotecas (não mexe)
- |— public/ ← arquivos servidos como estão
- |— index.html ← só tem a <div id="root">
- |— **src/** ← **o NOSSO código**
 - |— main.jsx · App.jsx · App.css
 - |— **components/** ← **um componente por arquivo**
 - |— data/ ← os dados (cardapio.js)

public/ × src/

● **public/** — arquivos que vão para o site do jeito que estão.

● /favicon.ico → acessado direto pela URL

● **src/** — o código que o Vite processa.

● components/ · data/ · assets/ · App.jsx

💡 Imagem usada dentro de componente → **src/assets/**

💡 Ícone da aba do navegador → **public/**

Como os arquivos se conectam

1. O navegador abre o **index.html** (a div root está vazia).
2. O index.html carrega o **main.jsx**.
3. O main.jsx desenha o **◁App />** dentro da div root.
4. O App importa e usa os **componentes**.
5. Cada componente devolve o seu pedaço da tela.

 **index.html** → **main.jsx** → **App.jsx** → **components/**

import e export

Export default: o principal do arquivo (só 1) → sem chaves.

- export default CardPrato;
- import CardPrato from "../components/CardPrato";

Export nomeado: pode ter vários → com chaves e mesmo nome.

- export const cardapio = [...];
- import { cardapio } from "../data/cardapio";

💡 Hoje vamos criar 3 componentes novos: todos com export default.

Componentização

Componentização = dividir a tela em partes, cada parte um componente.

App

- |— Header
- |— SecaoCardapio (uma por categoria)
 - | — CardPrato (um por prato)
 - | — Selo (Destaque, Vegetariano, Esgotado)
- Rodape

Componente: as regras

- `function Selo({ texto }) {`
 - `return <span className="selo">{texto}</span>;`
 - `}`
 - `export default Selo;`
-
- ✓ Função com nome em **Maiúscula**.
 - ✓ O return devolve **um** elemento por fora.
 - ✓ Um componente por arquivo, em **src/components/**.
 - ✓ export default no final.

Reutilizar um componente

Escreve **uma vez**, usa quantas vezes quiser, com dados diferentes:

- `<Selo texto="Destaque" tipo="destaque" />`
- `<Selo texto="Vegetariano" tipo="veg" />`
- `<Selo texto="Esgotado" tipo="esgotado" />`

- ✓ Mesmo código, três resultados diferentes na tela.
- ✓ Mudou o visual do selo? Muda em **um** lugar só.

Props: do pai para o filho

O pai escreve atributos na tag:

- `<Selo texto="Vegetariano" tipo="veg" />`

O React junta tudo em um objeto:

- `{ texto: "Vegetariano", tipo: "veg" }`

O filho recebe esse objeto no parâmetro:

- `function Selo({ texto, tipo }) { ... }`

- ✓ `{ texto, tipo }` pega cada valor pelo nome.

Props: tipos de valor

- texto="Pudim" ← **texto: aspas**
- preco={15} ← **número: chaves**
- vegetariano={true} ← **verdadeiro/falso: chaves**
- pratos={listaDePratos} ← **lista: chaves**
- onAdicionar={adicionar} ← **função: chaves**

✓ Regra: só texto vai entre aspas. O resto vai entre { }.

⚠ preco="15" chega como texto, não como número.

Props: valor padrão e só leitura

Valor padrão: vale quando o pai não manda a prop.

- `function Selo({ texto, tipo = "padrao" }) { ... }`
- `<Selo texto="Novo" />` → tipo vale "padrao"

Só leitura: o filho usa, mas não muda a prop.

- `texto = "outro";` //  errado

 Quem decide o valor é o pai.

Listas: `.map()` e `key`

`.map()` passa por cada item e devolve uma lista nova.

```
● {pratos.map((prato) => (  
●   <CardPrato key={prato.id} nome={prato.nome} />  
● ))}
```

✓ 6 pratos na lista = 6 cards na tela.

✓ `key` = valor que não se repete (`id`).

⚠ Sem `key`: aviso no console. Não use a posição (0, 1, 2).

Separar com `.filter()`

`.filter()` devolve uma lista nova só com quem passa na regra.

- `cardapio.filter(`
- `(prato) => prato.categoria === "Sobremesa"`
- `)`
- `// [ Pudim, Brigadeirão ]`

- ✓ A lista original não muda.
- ✓ Não precisa de estado: é só uma conta feita na hora.

Lista dentro de lista

```
● const categorias = ["Prato principal", "Sobremesa", "Bebida"];  
  
● {categorias.map((categoria) => (  
●   <SecaoCardapio key={categoria} titulo={categoria}  
●   pratos={cardapio.filter((p) => p.categoria === categoria)} />  
● )}}
```

✓ 1º .map: uma seção por categoria (no App).

✓ 2º .map: um card por prato (na SecaoCardapio).

Condicional: &&

&& = mostra SÓ SE a condição for verdadeira.

- {destaque && <Selo texto="Destaque" tipo="destaque" />}
- {vegetariano && <Selo texto="Vegetariano" tipo="veg" />}

- ✓ destaque é true → o selo aparece.
- ✓ destaque é false → não aparece nada.
- 💡 Use quando é "algo OU nada".

Condicional: ternário (? :)

? : = mostra UMA coisa OU outra.

- {disponivel ? (
- <button>Adicionar ao pedido</button>
-) : (
- <button disabled>Indisponível</button>
-)}

✓ Também serve para classe CSS:

- className={destaque ? "card-prato destaque" : "card-prato"}

Cuidados: o ! e o 0 na tela

! = não (inverte true e false).

● `{!disponivel && <Selo texto="Esgotado" tipo="esgotado" />}`

→ mostra o selo quando NÃO está disponível.

⚠ **Número antes do && aparece na tela:**

● `{pratos.length && <p>...</p>}` // mostra 0

● `{pratos.length > 0 && <p>...</p>}` // ✓

💡 Sempre escreva uma comparação.

React na web × React Native

O React é o mesmo. Muda só o que aparece na tela.

● **Igual:** componente, props, `.map()` + `key`, `&&` e ternário.

● web: `<div> <p> <button>` · mobile: `<View> <Text> <Pressable>`

● web: `onClick` · mobile: `onPress`

● web: `className` + CSS · mobile: `style` + `StyleSheet`

💡 Quem aprendeu um, já sabe a lógica do outro.

Live Code — cardápio

Ponto de partida: pasta livecoding-inicio (= final da aula 4).

1. Cardápio novo: 6 pratos + campos true/false.
2. Componente Selo + && (Destaque, Vegetariano).
3. Esgotado: ! e ternário no botão; destaque no className.
4. Componente SecaoCardapio (recebe lista por props).
5. Uma seção por categoria: .map + .filter.
6. Bônus: Rodape com valor padrão.

Ver **Guia de Live Coding** para o passo a passo.

Exercícios — cardápio

Pasta: exercicios/techfood-react (procure por TODO).

- **E1:** nova bebida no cardapio.js.
- **E2:** selo "Picante" com &&.
- **E3:** preço promocional com ternário.
- **E4:** quantidade no título: "Sobremesa (2)".
- **D1:** seção vazia mostra "Em breve!".
- 📁 **Entrega:** prints + código em 1 PDF (AULA05_SEUNOME.pdf).

Resumo



- ✓ Nosso código fica em `src/`; componentes em `src/components/`.
- ✓ Componentização: o que se repete vira componente.
- ✓ Props: o pai manda, o filho recebe e só lê.
- ✓ Só texto vai entre aspas; o resto entre `{ }`.
- ✓ Valor padrão: `{ tipo = "padrao" }`.
- ✓ `.map()` + key monta a lista; `.filter()` separa.
- ✓ `&&` = algo ou nada · `? :` = uma coisa ou outra.

A seguir (19/10): **estado e eventos — formulários controlados.**

Referências

REACT TEAM. Your First Component. Disponível em: react.dev/learn/your-first-component. Acesso em: out. 2026.

REACT TEAM. Passing Props to a Component. Disponível em: react.dev/learn/passing-props-to-a-component. Acesso em: out. 2026.

REACT TEAM. Rendering Lists. Disponível em: react.dev/learn/rendering-lists. Acesso em: out. 2026.

REACT TEAM. Conditional Rendering. Disponível em: react.dev/learn/conditional-rendering. Acesso em: out. 2026.

ANDRADE, Sidnei. Criação e interfaces para web HTML5 e CSS3. São Paulo: SENAI-SP, 2017.



ESCOLA SENAI ÍTALO BOLOGNA

Av. Goiás, 139

Telefone

(11) 2396-1999

Instagram

@senai.itu

Facebook

/senaisp.itu

Site

<https://sp.senai.br/unidade/itu/>